`jmap`: what is on the heap?

`jmap` attaches to a running JVM and prints its objects, class by class.

```
jmap -histo:live <pid>                          # reachable objects only (runs a full GC first)
jmap -histo <pid>                               # every object, garbage included
jmap -dump:live,format=b,file=heap.hprof <pid>   # full heap dump for an analyzer
```

- A histogram is cheap and small; a dump holds every object and field value.
- `:live` hides garbage, which is exactly what allocation tuning needs to see.

Reading a class histogram

```
 num     #instances         #bytes  class name (module)
-------------------------------------------------------
   1:         15430         698576  [B (java.base@25.0.4.1)
   2:          3302         399272  java.lang.Class (java.base@25.0.4.1)
   3:         15273         366552  java.lang.String (java.base@25.0.4.1)
```

- `#bytes` divided by `#instances` is the average object size: 24 B for `String`.
- Two histograms around a workload, subtracted, give the garbage it created.

References

- The `jmap` command (Java SE 25)
- Troubleshooting Guide: Diagnostic Tools (Java SE 25)